Despliegue público y migración de datos — La Migaja De San Carlos

Proyecto: sistema web de gestión integral para panadería y sucursales
Registro de trabajo: 27–28 de octubre de 2026 (hora de Guatemala / registros UTC del servidor)
Estado: aplicación publicada con HTTPS, datos locales migrados y pruebas funcionales satisfactorias.
Alcance: infraestructura Oracle Cloud, publicación de WAR Java, conexión Oracle Autonomous Database, dominio HTTPS, migración y validación. No incluye todavía la implementación de Firebase ni de Android Kotlin.

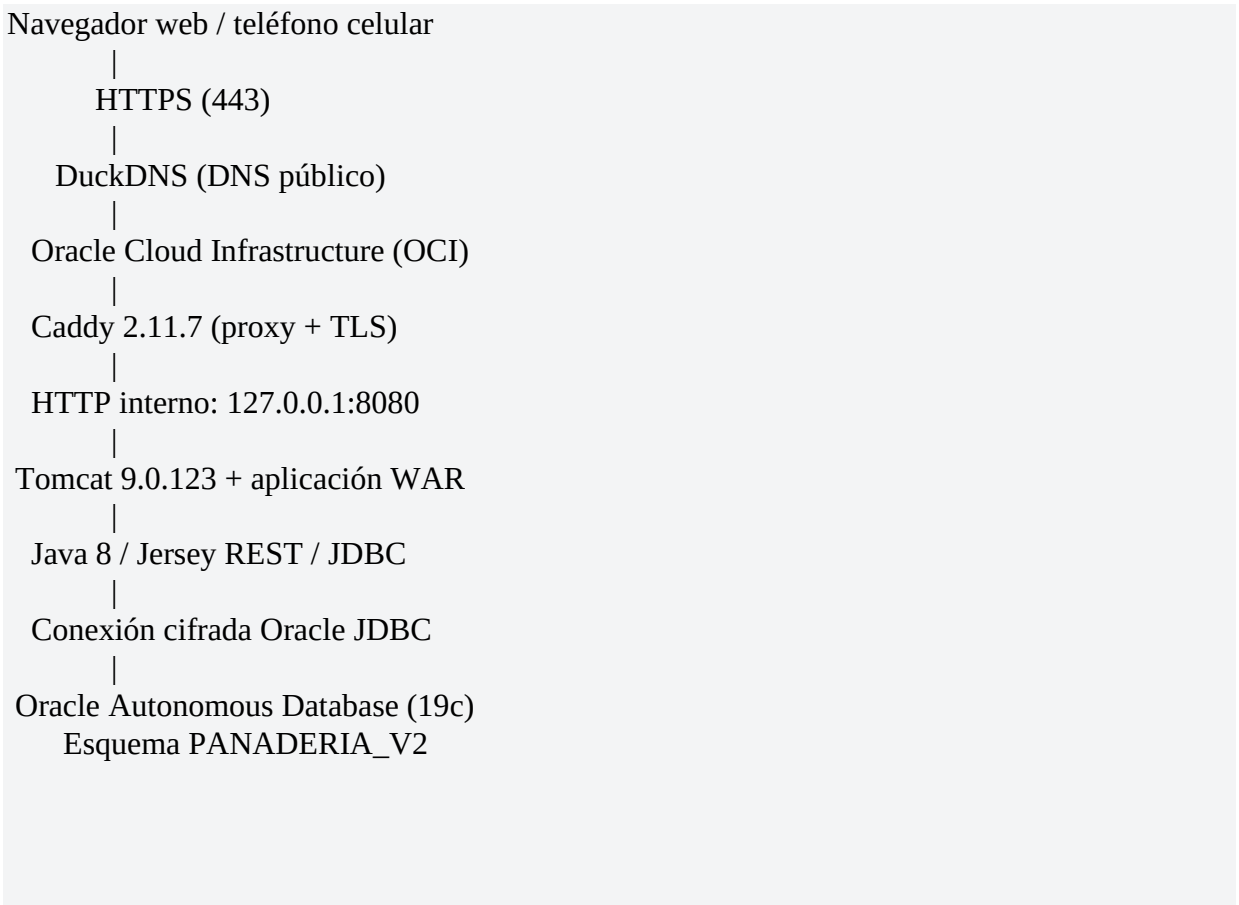
1. Objetivo y resultado

El objetivo de esta fase fue publicar en Internet el sistema que previamente funcionaba en NetBeans/Tomcat y Oracle XE local, sin depender de mantener encendida la computadora de desarrollo. Se desplegó el backend Java en una máquina virtual de Oracle Cloud, se conectó a una base de datos Oracle administrada, se activó HTTPS y se migraron los registros locales.

URL pública de la aplicación:
https://lamigajasancarlos.duckdns.org/Proyecto_PanaderiaSanCarlos/

La carga de la pantalla de inicio de sesión y el acceso autenticado se comprobaron desde un navegador de computadora y desde un teléfono celular. Después de migrar los datos, se verificó el acceso con los cinco usuarios existentes y la visualización de los módulos. También se creó un producto nuevo desde HTTPS para comprobar la escritura posterior a la migración.

2. Arquitectura implementada



Componente	Configuración aplicada
Máquina virtual	OCI Compute, Oracle Linux 8, Ampere A1 Flex, ARM64 (aarch64), 1 OCPU / 6 GB RAM; recurso elegido dentro de opciones Always Free elegibles
Servidor de aplicaciones	OpenJDK 8 y Apache Tomcat 9.0.123
Aplicación	Proyecto Maven empaquetado como Proyecto_PanaderiaSanCarlos.war
Backend	Java + Jersey REST, autenticación JWT, hashes de contraseñas BCrypt
Frontend	HTML, CSS y JavaScript, servido desde el mismo WAR
Base de datos	Oracle Autonomous Database, LaMigajaDB, servicio migajadb_low, esquema PANADERIA_V2
Proxy y HTTPS	Caddy 2.11.7 con certificados TLS automáticos
Dominio	lamigajasancarlos.duckdns.org administrado desde DuckDNS
Contexto de Tomcat	/Proyecto_PanaderiaSanCarlos/

Nota de costos: se escogieron recursos etiquetados como elegibles para *Always Free*. Su elegibilidad y los límites reales dependen de la cuenta, la región y el consumo; esta documentación no constituye una garantía de ausencia de cargos.

3. Preparación de Oracle Cloud

1. Se creó una red virtual (lamigaja-vcn) con subred pública e Internet Gateway.
2. Se creó una instancia de cómputo Oracle Linux 8 sobre Ampere ARM64 con dirección IPv4 pública.
3. Se configuró el acceso de administración por SSH con clave privada, sin utilizar contraseñas públicas ni subir la clave al repositorio.
4. En las reglas de entrada de la subred pública se permitieron TCP 80 (HTTP) y TCP 443 (HTTPS) desde Internet.
5. En el firewall del sistema operativo (firewalld) se habilitaron los servicios http y https.
6. Tomcat quedó enlazado a 127.0.0.1:8080: los puertos 8080 y 1521 no se abrieron como servicios públicos de la VM.

Comandos utilizados para el firewall del sistema operativo:

```
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --permanent --add-service=https
sudo firewall-cmd --reload
```

La administración SSH debe mantenerse restringida a orígenes confiables cuando sea viable.

4. Preparación de Tomcat y publicación del WAR

- Se instaló Java 8 y se configuró Tomcat 9.0.123 en /opt/tomcat.
- Se creó el servicio tomcat.service para que Tomcat funcione mediante systemd.
- Se generó el WAR desde el proyecto Maven de NetBeans y se transfirió al servidor mediante SCP/SSH.
- Antes de publicarlo se comprobó que el WAR incluía el controlador JDBC de Oracle y no incluía el archivo privado .env.

- En el WAR destinado a producción se retiraron dos controladores de prueba (PruebaBDController y PruebaController) antes de instalarlo. Pendiente preventivo: asegurar también en el código fuente que esos endpoints de prueba no vuelvan a incluirse al compilar versiones futuras.
- Se instaló el WAR en /opt/tomcat/webapps/Proyecto_PanaderiaSanCarlos.war.
- Se comprobó que Tomcat respondía internamente con HTTP 200 bajo el contexto de la aplicación.

Comandos útiles de administración y diagnóstico:

```
sudo systemctl status tomcat --no-pager
sudo journalctl -u tomcat -n 60 --no-pager
curl -I http://127.0.0.1:8080/Proyecto_PanaderiaSanCarlos/
```

Importante: las acciones anteriores describen el despliegue efectuado; el documento no reemplaza el procedimiento de compilación ni automatiza un nuevo despliegue.

5. Base de datos Oracle Cloud y configuración privada

Se aprovisionó una base de datos Oracle Autonomous Database en OCI y se configuró el esquema PANADERIA_V2, conservando el modelo relacional y los paquetes PL/SQL del proyecto. El esquema original incluye 15 tablas principales y 8 paquetes PL/SQL (con sus respectivos cuerpos).

La comunicación desde el servidor hacia Autonomous Database utiliza JDBC Oracle, conexión cifrada TLS, el host del servicio de OCI y el servicio migajadb_low.

La aplicación carga sus variables privadas desde un .env fuera del WAR y fuera del repositorio:

```
/opt/tomcat/.panaderia-san-carlos/.env
```

Nombres de variables utilizadas (solo nombres, no valores secretos):

```
PANADERIA_DB_URL=<URL_JDBC_PRIVADA>
PANADERIA_DB_USER=<USUARIO_DE_ESQUEMA>
PANADERIA_DB_PASSWORD=<CONTRASENA_NO_PUBLICAR>
PANADERIA_JWT_SECRET_BASE64=<SECRETO_JWT_NO_PUBLICAR>
```

El archivo privado se configuró con propiedad root:tomcat, permisos 640 y directorio contenedor con permisos restringidos. Se comprobó que el servicio Tomcat podía leer la configuración. También se configuró el acceso de red autorizado en Oracle Autonomous Database; **no se dejó públicamente expuesta una base de datos instalada en la VM**.

La conexión desde la aplicación se verificó con la API de autenticación y, posteriormente, con el inicio de sesión real y la consulta del catálogo.

6. Dominio DuckDNS y HTTPS con Caddy

Se registró el dominio lamigajasancarlos.duckdns.org y se asoció a la IP pública de la instancia OCI. La resolución se confirmó con una consulta DNS al resolvedor 1.1.1.1.

Caddy se instaló en Oracle Linux mediante el repositorio COPR de Caddy y dnf. La versión observada fue **2.11.7**.

Contenido esencial de /etc/caddy/Caddyfile:

```
lamigajasancarlos.duckdns.org {
    @raiz path /
    redir @raiz /Proyecto_PanaderiaSanCarlos/ 302

    reverse_proxy 127.0.0.1:8080
}
```

Caddy permite HTTPS público, redirige la raíz del dominio al contexto de la aplicación y reenvía internamente las solicitudes a Tomcat.

Comandos de validación/administración:

```
sudo caddy validate --config /etc/caddy/Caddyfile --adapter caddyfile
sudo systemctl enable --now caddy
sudo systemctl status caddy --no-pager
curl -I https://lamigajasancarlos.duckdns.org/Proyecto_PanaderiaSanCarlos/
```

Comprobación funcional: el usuario abrió la URL pública desde una computadora y pudo iniciar sesión desde un celular. Esto confirmó el acceso HTTPS y el funcionamiento de la aplicación desde fuera del entorno local de desarrollo.

7. Migración de Oracle XE local a Autonomous Database

Se migraron los datos operativos de la base local Oracle XE, manteniendo los catálogos que ya estaban inicializados en Cloud.

7.1 Respaldo y preparación

- Se exportó un respaldo SQL del esquema local desde Oracle SQL Developer y se guardó solo en la computadora.
- En Cloud se crearon cinco copias de datos previas a la migración: BKP_MIG_ROL, BKP_MIG_SUCURSAL, BKP_MIG_PRODUCTO, BKP_MIG_INVENTARIO y BKP_MIG_USUARIO.
- Se crearon 12 tablas auxiliares MIG_ mediante CREATE TABLE ... AS SELECT ... WHERE 1=0.
- Desde SQL Developer se exportaron solo los datos de las 12 tablas transaccionales y de catálogo seleccionadas, con formato INSERT.
- Se cambió el destino de las instrucciones exportadas para cargar los registros en PANADERIA_V2.MIG_* sin modificar todavía las tablas definitivas.
- Se comprobaron 130 instrucciones de inserción y, tras ejecutarlas, 130 registros en las tablas auxiliares, distribuidos como sigue:

Tabla de negocio	Registros locales verificados
USUARIO	5
PRODUCTO	21
INVENTARIO	21
RECETA	4
DETALLE_RECETA	7
MOVIMIENTO_INVENTARIO	27
PEDIDO	9
DETALLE_PEDIDO	10
PRODUCCION	8
DETALLE_PRODUCCION	9
DESPACHO	4
DETALLE_DESPACHO	5

Tabla de negocio	Registros locales verificados
Total	**130**

No se migraron las entradas de TOKEN_JWT_REVOCADO, dado que son registros temporales de control de sesiones. Los roles y las sucursales ya coincidían en ambas bases.

7.2 Resolución de conflictos

La comparación mostró que los **19 productos iniciales** tenían los mismos identificadores y atributos; se agregaron únicamente dos productos locales nuevos (PAN010 y PAN011). Las existencias de inventario en Cloud se actualizaron con los valores locales, manteniendo el historial de movimientos.

El principal conflicto fue el identificador del administrador: la cuenta de demostración existía en ambas bases con **IDs distintos**, mientras que el ID 1 de Cloud ya estaba ocupado. La integración conservó la cuenta de demostración de Cloud y ajustó las claves foráneas hacia los usuarios correctos.

Identidad lógica	ID original local	ID destino Cloud
Administrador de demostración	41	1 (conservado)
Administrador local	1	42
Usuario sucursal central	2	2
Usuario sucursal Chiva	21	21
Usuario de producción	22	22

El script de integración incorporó las tablas relacionadas en un orden compatible con sus claves foráneas. Contenía validaciones de cantidades y de estado previo, un COMMIT al concluir correctamente y un ROLLBACK en caso de excepción.

7.3 Resultado de la integración

La salida registrada en Database Actions fue:

```
INTEGRACION COMPLETA: 130 registros en tablas definitivas. Sincronizar IDENTITY.
Procedimiento PL/SQL terminado correctamente.
```

Interpretación precisa: los 130 corresponden a los registros finales reconciliados de las 12 tablas, antes de la prueba de creación. No significa que se insertaran 130 filas nuevas en las tablas definitivas: algunos productos y el usuario de demostración ya existían, y los inventarios existentes se actualizaron.

7.4 Sincronización de identificadores

Las 12 tablas usan columnas de identidad GENERATED BY DEFAULT AS IDENTITY. Después de la integración, se sincronizaron sus generadores con los máximos existentes mediante la forma:

```
ALTER TABLE NOMBRE_TABLA
MODIFY (NOMBRE_ID GENERATED BY DEFAULT AS IDENTITY (START WITH
LIMIT VALUE));
```

Las 12 alteraciones fueron reportadas como correctas. Esto se realizó **antes** de crear un nuevo registro desde HTTPS, para reducir el riesgo de reutilización de IDs.

8. Validación y evidencias funcionales

Comprobación	Resultado observado o confirmado
Acceso público HTTPS	Pantalla de login cargada desde el dominio DuckDNS
Acceso desde celular	Inicio de sesión correcto desde teléfono celular
Conexión JDBC a Autonomous Database	Login y consulta de datos desde el despliegue en Cloud
Acceso de usuarios	Inicio de sesión correcto de los **5 usuarios** existentes
Productos después de migración	**21 productos** visibles
Inventario	Existencias locales visibles; p. ej., Frances 20, harina de fuerza 34.5, harina suave 20, azúcar 91
Módulos operativos	Se reportó visualización correcta de pedidos, recetas, producción, movimientos y despachos
Escritura posterior a migración	Se creó desde HTTPS el producto de prueba PAN012 («Pan prueba cloud»)
Productos después de la prueba	**22 productos** visibles

Se probó la creación de un producto nuevo para verificar la escritura y la generación de un nuevo ID_PRODUCTO interno; el código comercial PAN012 fue introducido manualmente en el formulario, como establece la aplicación.

Estas son **pruebas manuales realizadas durante el despliegue**. No sustituyen una suite automatizada ni una auditoría integral de seguridad.

9. Separación de entornos

Entorno local	Entorno publicado
NetBeans / Tomcat local	Tomcat y Caddy en OCI
http://localhost:8080/Proyecto_PanaderiaSanCarlos/	https://lamigajasancarlos.duckdns.org/Proyecto_PanaderiaSanCarlos/
Oracle XE / XEPDB1	Oracle Autonomous Database / LaMigajaDB
Desarrollo y pruebas	Demostración y acceso remoto

No existe sincronización automática bidireccional entre Oracle XE local y Autonomous Database. Los registros nuevos creados en Cloud no aparecen automáticamente en local, ni viceversa. Para actualizar la versión publicada del código, es necesario compilar y desplegar un nuevo WAR.

10. Seguridad y recomendaciones antes de la exposición

- **Nunca subir a Git** archivos .env reales, claves SSH (*.key, *.pem), tokens DuckDNS, contraseñas, wallets Oracle ni respaldos/exportaciones SQL con datos reales.
- Mantener el repositorio sin endpoints de diagnóstico y verificar las rutas publicadas antes de cada nuevo WAR.
- Mantener Tomcat solo en 127.0.0.1:8080, usando Caddy como entrada HTTPS pública.

- Restringir el acceso administrativo SSH y los orígenes autorizados en la base de datos cuando sea posible.
- Evitar credenciales de demostración débiles y revisar permisos y manejo de tokens JWT.
- Crear y probar un procedimiento de **respaldo recuperable de Cloud**. Las tablas BKP_MIG_* guardan copias de datos previas a la migración, **pero no constituyen un respaldo externo completo**.
- Antes de la demostración, verificar que la VM, la base de datos, el dominio y el certificado HTTPS estén disponibles.
- Monitorear las cuotas/consumos de OCI y conservar recursos dentro de los límites aplicables de Always Free.
- Si se cambia la IP pública al recrear la VM, actualizar el registro en DuckDNS y revisar las reglas de acceso correspondientes.
- Las tablas de trabajo MIG_* y BKP_MIG_* quedan como evidencia técnica temporal; eliminarlas, si se decide, requiere respaldo y autorización posteriores.

11. Consultas y comandos de verificación

Servidor OCI (por SSH):

```
sudo systemctl status tomcat --no-pager
sudo systemctl status caddy --no-pager
curl -I http://127.0.0.1:8080/Proyecto_PanaderiaSanCarlos/
curl -I https://lamigajasancarlos.duckdns.org/Proyecto_PanaderiaSanCarlos/
sudo journalctl -u caddy -n 50 --no-pager
```

Oracle Cloud (consulta de solo lectura):

```
SELECT COUNT(*) AS USUARIOS FROM USUARIO;
SELECT COUNT(*) AS PRODUCTOS FROM PRODUCTO;
SELECT COUNT(*) AS PEDIDOS FROM PEDIDO;
SELECT COUNT(*) AS PRODUCCIONES FROM PRODUCCION;
SELECT COUNT(*) AS DESPACHOS FROM DESPACHO;
```

Los recuentos pueden aumentar después de la migración por nuevas operaciones realizadas desde la aplicación.